4.3 Bag of Words

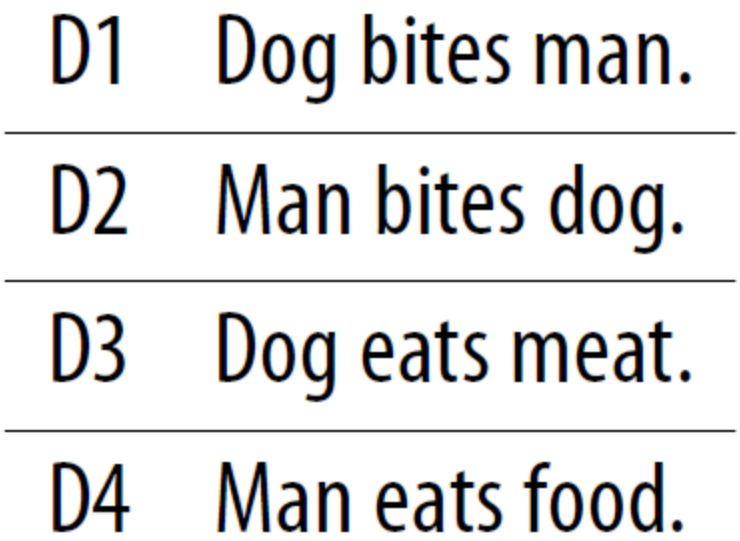
Count tokens and ignore order.

1. Count the words, drop the order

Bag of words (BoW) is the workhorse representation for classical text classification. You treat the document as a **collection** of tokens. Order and syntax go away. What remains is "which words appeared, and how often."

The intuition: a class (spam, sports, negative review) has a characteristic vocabulary. If you count those words, a linear model can often find the class.

Same toy corpus as notes **4.1–4.2**:



Ids: dog = 1, bites = 2, man = 3, meat = 4, food = 5, eats = 6.

Each document becomes one vector of length (|V|). Entry (i) is the number of times word (i) occurs.

Doc	Text	BoW
(D_1)	dog bites man	`[1 1 1 0 0 0]`
(D_2)	man bites dog	`[1 1 1 0 0 0]`
(D_3)	dog eats meat	`[1 0 0 1 0 1]`
(D_4)	man eats food	`[0 0 1 0 1 1]`

(D_1) and (D_2) are **identical**. The bite went the other way. BoW cannot see that.

2. Why people keep using it

It is simple. It gives a **fixed-length** vector for a sentence of any length. That is the main upgrade from one-hot sequences.

Documents that share vocabulary land near each other. Euclidean distance (D_1) to (D_2) is 0. Distance (D_1) to (D_4) is 2. The space is a crude semantic map: similar word sets, similar vectors.

For topic or spam, "crude" is often enough. A logistic regression or linear SVM on BoW (or TF–IDF, note **4.5**) is still a strong baseline. Build it before you reach for a network.

In code you will use `sklearn.feature_extraction.text.CountVectorizer`. Fit on the training corpus only. Transform the test set with the same vocabulary.

---

3. What it cannot do

Sparsity grows with (|V|). A news crawl has hundreds of thousands of types. Most entries in any one document vector are zero. A common fix is to keep only the (n) most frequent words, or words above a document-frequency cutoff.

No notion of synonymy. "I run", "I ran", and "I ate" are equally far apart. *run* and *ran* do not share a coordinate.

OOV. A word that never appeared in the training corpus has no column. The vectorizer drops it or maps it to an unknown bucket if you built one. The default is to drop it.

Order is gone. That is the name. (D_1) and (D_2) collapse. Negation ("not good") is easy to miss if *not* and *good* are just two counts.

---

4. Practical knobs

Knob	What it does
Lowercase, strip punctuation	Same token, one column
Stop-word list	Drops *the*, *a*, *of*
`min_df` / `max_df`	Drops rare noise and corpus-wide glue words
`max_features`	Hard cap on (
Binary BoW	1 if present, ignore count

Binary BoW is enough when presence matters more than frequency (short tweets, many function words already removed).

None of these knobs restore word order. That is note **4.4**.

Lab habit. Fit the vectorizer on the training documents only, then `transform` the validation and test sets. If you `fit` on everything, rare test words leak into the vocabulary and your score is a lie. The

same rule applies to TF–IDF in note **4.5**.

A sanity check on the toy set: after `CountVectorizer`, (D_1) and (D_2) must be the same row. If they are not, you accidentally kept order (you built n-grams or you tokenized the two sentences differently).

---

5. Cosine on the toy set

(D_1 = D_2 = [1,1,1,0,0,0]). Cosine is 1. (D_3 = [1,0,0,1,0,1]) shares only *dog* with (D_1). The two vectors are not opposites; they just overlap in one coordinate. That is all BoW "semantics" is: shared types.

If you L2-normalize each row, Euclidean nearest neighbors and cosine nearest neighbors agree. Many sklearn pipelines do that normalize without saying so. Know whether you turned it on.

---

6. Practice

1. Write the BoW vector for "dog bites dog" with the id table above.
2. A reviewer writes "not good." Another writes "good." Why might a unigram bag confuse a sentiment model?
3. You have 50,000 training documents and a vocabulary of 200,000 types. Name two CountVectorizer settings you would change before fitting a linear classifier.